BILOW

Sida loo bartaa

Jadwalka 28 usbuuc

A · AASAASKA

0. Diyaarinta

1. Aasaaska Java

2. OOP

3. Java horumarsan

4. SQL iyo Database

5. JDBC

6. Git iyo GitHub

7. Maven

B · SPRING BOOT

8. Bilaabka NutriTrack

9. Database iyo JPA

10. REST API

11. Validation, Error, Log

12. JWT Security

13. Testing

C · PRODUCTION

14. Docker iyo Linux

15. CI/CD

16. Deployment

D · XIRFAD

17. Patterns iyo Architecture

18. Portfolio iyo Shaqo

KOORSADA BUUXDA · AF-SOOMAALI

Java Developer: Eber ilaa Deployment

Koorsadan waxay ku kaxaysaa tallaabo-tallaabo, laga bilaabo qoraalka Java-gaaga ugu horreeya ilaa mashruuc Spring Boot ah oo live ah oo leh domain iyo SSL. Mashruuca la dhisayo waa **NutriTrack**, API cuntada, kalooriyada iyo biyaha la raaco.

19	~28	1	Java 21
module	usbuuc	mashruuc buuxa	iyo Spring Boot 3

0 / 19 dhammaaday

BILOW

Sida loo bartaa koorsadan

Developer ma noqdo adigoo daawanaya muuqaallo oo kaliya. Waxaad noqotaa markaad code qorto, jabiso, oo hagaajiso. Raac xeerarkan:

- **Qor code-ka gacantaada.** Ha koobiyeyn oo dhegsan. Qor xaraf-xaraf.
- **Maalin kasta 2–3 saacadood.** Joogto waa ka muhiimsan tahay saacado badan oo mar-mar ah.
- **Module kasta layli ku dhammee.** Layliga ayaa kuu sheegaya inaad fahantay iyo in kale.
- **GitHub ku rid maalin kasta.** Profile-kaaga ayaa noqonaya CV-gaaga.
- **Marka aad ku dhegto 30 daqiiqo**, soo qor su'aasha si cad (error-ka buuxa, code-ka, waxa aad filaysay). Taasi lafteeda badanaa waxay muujisaa jawaabta.

Checkbox-ka *Waan dhameeyay* ee hoos kasta wuxuu ku xafidaa horumarkaaga browser-kan.

JADWAL

Jadwalka 28 usbuuc

MODULE	MOWDUUC	MUDDA
0	Diyaarinta qalabka	3 maalmood
1	Aasaaska Java	3 usbuuc

2	OOP	3 usbuuc
3	Java horumarsan	4 usbuuc
4–5	SQL, Database, JDBC	3 usbuuc
6–7	Git, GitHub, Maven	1 usbuuc
8–10	Spring Boot, JPA, REST API	4 usbuuc
11–12	Validation, Error, JWT Security	3 usbuuc
13	Testing	1.5 usbuuc
14–16	Docker, CI/CD, Deployment	3 usbuuc
17–18	Patterns, Portfolio, Shaqo raadin	1.5 usbuuc

Haddii aad maalin kasta haysato 4 saacadood, waxaad dhammayn kartaa 5–6 bilood gudahood. Haddii aad haysato 1–2 saacadood, 9–10 bilood ayaa ku filan.

QAYBTA A · AASAASKA · 3 MAALMOOD

0. Diyaarinta qalabka

Hadaf: kombiyuutarkaaga oo diyaar u ah Java, adigoo orda program ugu horreeya.

KU RAKIB

- **JDK 21** (LTS). Isticmaal Temurin ama Oracle JDK. Hubi: `java -version`
- **IntelliJ IDEA Community** (bilaash) ama VS Code oo leh Extension Pack for Java.
- **Git** iyo koonto **GitHub**.
- **Docker Desktop** (waxaad u baahan doontaa Module 9 kadib).
- **Postman** ama Bruno, si aad API u tijaabiso.

TERMINAL-KA (AASAAS)

```
pwd              # halka aad joogto
ls               # liiska faylasha
cd projects      # gal folder
mkdir java-lab   # samee folder
```

```
rm file.txt      # tir fayl
java -version
javac -version
```

> **LAYLI**
>
> - Samee folder `java-lab` . Ka gudaha qor `Hello.java` , ku orod `javac Hello.java && java Hello` .
> - Samee repository GitHub oo la yiraahdo `java-journey` .

⬛ Waan dhameeyay

QAYBTA A · AASAASKA · 3 USBUUC

1. Aasaaska Java

Hadaf: inaad qorto program yar oo isticmaala doorsoomayaal, shuruudo, loop-yo iyo methods.

WAXAAD BARANAYSAA

- JDK, JRE, JVM: waxa ay yihiin
- Variables iyo data types (`int` , `double` , `boolean` , `char` , `String`)
- Operators
- `if` / `else` , `switch`
- `for` , `while` , `do-while`
- Methods iyo parameters
- Arrays iyo `String` methods
- `Scanner` (gelinta xogta)

JAVA CASRIGA AH

- `var`
- Text blocks
- Switch expressions
- Records (Module 3)

```
import java.util.Scanner;

public class Bmi {
    static double calculate(double kg, double meters) {
        return kg / (meters * meters);
    }
```

```
    }

    static String category(double bmi) {
        if (bmi < 18.5) return "Miisaan yar";
        else if (bmi < 25) return "Caadi";
        else if (bmi < 30) return "Miisaan badan";
        return "Buurnaan";
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Miisaanka (kg): ");
        double kg = in.nextDouble();
        System.out.print("Dhererka (m): ");
        double m = in.nextDouble();

        double bmi = calculate(kg, m);
        System.out.printf("BMI = %.1f (%s)%n", bmi, category(bmi));
    }
}


// Switch expression (Java 14+)
String day = switch (n) {
    case 1, 2, 3, 4, 5 -> "Maalin shaqo";
    case 6, 7 -> "Fasax";
    default -> "Aan sax ahayn";
};

// Array iyo loop
int[] marks = {70, 85, 92, 60};
int sum = 0;
for (int mk : marks) sum += mk;
System.out.println("Celceliska: " + (double) sum / marks.length);
```

LAYLI

- Calculator console ah oo isticmaala `switch` (+, -, *, /).
- Ciyaar "Qiyaas lambarka": kombiyuutarku wuxuu doortaa 1–100 (`Random`), adigu waad qiyaasaysaa.
- Function: `isPrime(int n)` iyo `fibonacci(int n)` .
- Qor liis 10 lambar ah oo la ogaado ugu weyn, ugu yar, iyo celceliska.

HUBI

- Ma sharixi kartaa farqiga `int` iyo `double` iyo sababta `5 / 2` u dhigmo 2.2

- Ma sharxi kartaa farqiga `int` iyo `double`, iyo sababta `5 / 2` u dhigmo `2`?
- Ma garanaysaa farqiga `==` iyo `.equals()` ee String-ka?

⬛ Waan dhameeyay

QAYBTA A · AASAASKA · 3 USBUUC

2. Object Oriented Programming (OOP)

Hadaf: inaad mashruuc yar u habayso classes, objects iyo xiriir dhexdooda ah.

AFARTA TIIR

- **Encapsulation:** xogta qari (`private`), ka bixi getters/setters ama methods
- **Inheritance:** `extends`
- **Polymorphism:** method isku magac ah oo ku shaqeeya siyaabo kala duwan
- **Abstraction:** `abstract class` iyo `interface`

KALE

- Constructors, `this`, `static`, `final`
- `equals()`, `hashCode()`, `toString()`
- Enums
- Packages iyo access modifiers
- Composition vs inheritance

```
public interface Payable {
    double amount();
}

public abstract class Account implements Payable {
    private final String owner;
    protected double balance;

    protected Account(String owner, double balance) {
        this.owner = owner;
        this.balance = balance;
    }

    public void withdraw(double value) {
        if (value <= 0 || value > balance)
            throw new IllegalArgumentException("Qadarka aan sax ahayn");
        balance -= value;
    }
```

```
    public String getOwner() { return owner; }
    public abstract double monthlyFee();
}

public class SavingsAccount extends Account {
    public SavingsAccount(String owner, double balance) { super(owner, balance);
    @Override public double monthlyFee() { return 0; }
    @Override public double amount() { return balance; }
}

public class CurrentAccount extends Account {
    public CurrentAccount(String owner, double balance) { super(owner, balance);
    @Override public double monthlyFee() { return 2.5; }
    @Override public double amount() { return balance; }
}
```

Polymorphism: `List<Account> all = List.of(new SavingsAccount("Axmed", 100), new CurrentAccount("Faadumo", 50));` kadibna loop ku samee `acc.monthlyFee()`. Mid walba wuxuu siinayaa jawaabtiisa.

> **MASHRUUC YAR: LIBRARY MANAGEMENT (CONSOLE)**
>
> - Classes: `Book`, `Member`, `Library`, `Loan`.
> - Xeerar: xubin kaliya 3 buug ayuu amaahan karaa; buug la amaahday dib looma amaahin karo.
> - Isticmaal `enum BookStatus { AVAILABLE, BORROWED }` iyo exception gaar ah.
> - Ku rid GitHub oo leh `README.md`.

HUBI

- Goorma baa la doortaa `interface` halkii `abstract class`?
- Maxay tahay sababta fields-ka loo dhigo `private`?

☐ Waan dhameeyay

QAYBTA A · AASAASKA · 4 USBUUC

3. Java horumarsan

Hadaf: inaad si fiican u isticmaasho Collections, Streams, Exceptions iyo files, kuna bilowdo threads.

Collections iyo Generics

```
List<String> names = new ArrayList<>();
names.add("Maryan");
names.add("Cali");

Map<String, Integer> stock = new HashMap<>();
stock.put("Bariis", 20);
stock.merge("Bariis", 5, Integer::sum);   // 25

Set<String> unique = new HashSet<>(names);
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.pop();
```

Baro goorta la isticmaalo `ArrayList` vs `LinkedList`, `HashMap` vs `TreeMap`, iyo sida `hashCode/equals` u saameeyaan `HashMap`.

Exceptions

```
public class InsufficientStockException extends RuntimeException {
    public InsufficientStockException(String item) {
        super("Kaydka " + item + " ma filna");
    }
}

try (var reader = Files.newBufferedReader(Path.of("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) System.out.println(line);
} catch (IOException e) {
    System.err.println("Fayl lama akhrin karo: " + e.getMessage());
}
```

Records, Lambdas iyo Streams

```
record Product(String name, String category, double price) {}

List<Product> products = List.of(
    new Product("Caano", "Cunto", 2.5),
    new Product("Bariis", "Cunto", 18.0)
```

```
    new Product("Baalis", "Cunto", 18.0),
    new Product("Saabuun", "Guri", 1.2));

double total = products.stream()
    .filter(p -> p.category().equals("Cunto"))
    .mapToDouble(Product::price)
    .sum();

Map<String, List<String>> byCategory = products.stream()
    .collect(Collectors.groupingBy(Product::category,
             Collectors.mapping(Product::name, Collectors.toList())));

Optional<Product> cheapest = products.stream()
    .min(Comparator.comparingDouble(Product::price));
```

Multithreading (aasaas)

```
ExecutorService pool = Executors.newFixedThreadPool(4);
List<Future<Integer>> results = new ArrayList<>();
for (int i = 1; i <= 4; i++) {
    final int n = i;
    results.add(pool.submit(() -> n * n));
}
for (var f : results) System.out.println(f.get());
pool.shutdown();
```

Ogow: `synchronized` , `AtomicInteger` , `CompletableFuture` , iyo in Java 21 ay leedahay virtual threads.

> **LAYLI**
>
> - **Contact Manager:** ku keydi xiriirada `List` , raadi magac ahaan, ku keydi fayl CSV, markale akhri.
> - Fayl qoraal ah (sheeg 1000 erey): tiri erey kasta imisa jeer ayay ka muuqataa (`Map` + Streams), ku soo bandhig 10-ka ugu badan.
> - Samee 3 task oo isku mar ku socda `ExecutorService` .

Waan dhameeyay

OAYBTA A · AASAASKA · 2 USBUUC

4. SQL iyo Database

Hadaf: inaad naqshadayso tables xiriir leh oo aad qorto su'aalo (queries) adag.

Isticmaal **PostgreSQL** (waa kan mashruucu isticmaalayo). Ku orod Docker:

```
docker run --name pg -e POSTGRES_PASSWORD=secret -p 5432:5432 -d postgres:16-alp
```

```
CREATE TABLE students (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL
);

CREATE TABLE courses (
  id BIGSERIAL PRIMARY KEY,
  title VARCHAR(120) NOT NULL
);

CREATE TABLE enrollments (          -- many-to-many
  student_id BIGINT REFERENCES students(id) ON DELETE CASCADE,
  course_id  BIGINT REFERENCES courses(id),
  grade INT CHECK (grade BETWEEN 0 AND 100),
  PRIMARY KEY (student_id, course_id)
);

SELECT s.name, c.title, e.grade
FROM enrollments e
JOIN students s ON s.id = e.student_id
JOIN courses  c ON c.id = e.course_id
WHERE e.grade >= 50
ORDER BY e.grade DESC;

SELECT c.title, COUNT(*) AS arday, ROUND(AVG(e.grade), 1) AS celcelis
FROM enrollments e JOIN courses c ON c.id = e.course_id
GROUP BY c.title
HAVING COUNT(*) > 5;
```

BARO

- `SELECT` / `INSERT` / `UPDATE` / `DELETE`
- `JOIN` (INNER, LEFT)

MUHIIM

- **Index:** waxay dedejisaa raadinta, waxayse hoos u dhigtaa qoraalka

- `GROUP BY`, `HAVING`
- Primary key, foreign key, unique, check
- Relationships: 1-1, 1-N, N-N
- **Transaction:** `BEGIN` / `COMMIT` / `ROLLBACK`
- Normalization (1NF–3NF)
- `EXPLAIN` si aad u aragto qorshaha query-ga

> **LAYLI**
>
> - Naqshadee database **School**: students, teachers, courses, enrollments, attendance. Sawir ERD ah (dbdiagram.io).
> - Qor 15 query: ardayda sare, macallinka ugu arday badan, ardayda aan imaan, iwm.

☐ Waan dhameeyay

QAYBTA A · AASAASKA · 1 USBUUC

5. JDBC

Hadaf: inaad fahanto sida Java ula hadasho database-ka, kahor intaadan isticmaalin framework.

Spring Data waxay si otomaatig ah u qaadaysaa shaqadan, laakiin haddii aad aragto JDBC, waad fahmi doontaa waxa hoos socda.

```
String url = "jdbc:postgresql://localhost:5432/school";

try (Connection con = DriverManager.getConnection(url, "postgres", "secret");
     PreparedStatement ps = con.prepareStatement(
         "SELECT id, name FROM students WHERE name ILIKE ?")) {

    ps.setString(1, "%ali%");
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getLong("id") + " " + rs.getString("name"));
        }
    }
}
```

Ka digtoonow SQL Injection: weligaa ha ku dhejin input-ka isticmaalaha string-ka SQL. Isticmaal `PreparedStatement` iyo `?`.

> **LAYLI**
>
> - Samee `StudentDao` leh `save`, `findById`, `findAll`, `update`, `delete`. Console menu ayaa isticmaalaya.

☐ Waan dhameeyay

QAYBTA A · AASAASKA · 3 MAALMOOD

6. Git iyo GitHub

Hadaf: inaad si nidaamsan u maamusho code-kaaga oo aad la shaqayso kuwa kale.

```
git init
git add .
git commit -m "feat: add food entity"
git branch -M main
git remote add origin https://github.com/USERNAME/nutritrack.git
git push -u origin main

git switch -c feature/auth      # branch cusub
git switch main
git merge feature/auth
git pull
git log --oneline --graph
git stash                       # kaydi isbeddelka ku meel gaar ah
```

- Qor `.gitignore` (`target/`, `.idea/`, `.env`). **Weligaa ha ku rin furaha sirta ah ama password-ka GitHub.**
- Isticmaal fariimo commit oo cad: `feat:`, `fix:`, `docs:`, `test:`.
- Ku shaqee branch kasta feature; fur **Pull Request**, naftaada dib u eeg, kadibna merge.
- Baro sida loo xaliyo **merge conflict**.

> **LAYLI**

- Samee conflict ala-ka-sameysan (laba branch ayaa isla xariiq beddela), xalli.
- Fur PR mashruuca Library-gaaga.

☐ Waan dhameeyay

QAYBTA A · AASAASKA · 3 MAALMOOD

7. Maven

Hadaf: inaad fahanto `pom.xml` iyo wareegga build-ka.

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-lang3</artifactId>
  <version>3.14.0</version>
</dependency>
```

```
mvn clean          # tir target/
mvn compile
mvn test
mvn package        # samee .jar
mvn verify         # test + integration checks
mvn dependency:tree
```

- Fahan: `groupId` , `artifactId` , `version` , `scope` (compile, test, runtime).
- Gradle waa beddel; hadda ku dheji Maven, maxaa yeelay koorsadan oo dhan way isticmaashaa.

> **LAYLI**
>
> - U beddel mashruuca Library Maven project, ku dar JUnit, oo samee `mvn package` . Orod `java -jar target/...jar` .

☐ Waan dhameeyay

QAYBTA B · SPRING BOOT · 1 USBUUC

8. Bilaabka mashruuca NutriTrack

> **Hadaf:** inaad abuurto mashruuc Spring Boot oo ku shaqeeya PostgreSQL oo leh qaab-dhismeed nadiif ah.

Waxa la dhisayo

NutriTrack waa REST API oo isticmaale kastaa ku diwaangaliyo cuntada uu cunay, biyaha uu cabbay, una arko soo koobid maalinle ah (kalooriyo, protein, carbs, fat). Waxa ku jira:

- Diiwaangelin iyo login (JWT), doorar `USER` iyo `ADMIN`
- Liiska cuntooyinka (raadin + pagination); ADMIN ayaa kordhin/tirin kara
- Diiwaanka cuntada maalinlaha ah iyo biyaha
- Soo koobid maalinle ah
- Validation, error response nadiif ah, Flyway, tests, Docker, CI/CD, deployment

Spring Core: waxa aad hore u fahmaysaa

- **Dependency Injection:** Spring ayaa abuura objects (beans) oo ku siiya meesha loo baahan yahay.
- **IoC Container:** waa meesha ay beans-ku ku nool yihiin.
- Annotations: `@Component` , `@Service` , `@Repository` , `@RestController` , `@Configuration` , `@Bean` .

```
@Service
@RequiredArgsConstructor          // Lombok: constructor injection
public class FoodService {
    private final FoodRepository foodRepository;   // Spring ayaa soo gelinaya
}
```

Samee mashruuca

Tag **start.spring.io**: Maven, Java 21, Spring Boot 3.x (kan ugu dambeeya ee la taageeray), Packaging Jar, Group `com.nutritrack` . Ku dar dependencies:

- Spring Web, Spring Data JPA, Validation, Spring Security
- PostgreSQL Driver, Flyway Migration, Lombok, Spring Boot Actuator
- Testcontainers (tiiaabo ahaan)

- Testcontainers (tijaabo ahaan)

Ku dar gacanta JWT iyo Swagger UI:

pom.xml

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>
```

Haddii Flyway uu ka cabanayo PostgreSQL, hubi inuu jiro dependency-ga `flyway-database-postgresql` (start.spring.io ayaa badanaa ku darta).

Qaab-dhismeedka packages

```
com.nutritrack
├── NutriTrackApplication.java
├── config/          SecurityConfig, OpenApiConfig
├── controller/      AuthController, FoodController, EntryController, WaterContr
├── dto/             records: request / response
├── entity/          User, Food, FoodEntry, WaterLog, Role, MealType
├── exception/       ResourceNotFoundException, GlobalExceptionHandler
├── repository/      Spring Data interfaces
├── security/        JwtService, JwtAuthFilter, CustomUserDetailsService
└── service/         AuthService, FoodService, EntryService, SummaryService
```

Qaabkan waxaa la yiraahdaa **Layered Architecture**: Controller → Service → Repository → Database. Controller kaliya wuxuu la hadlaa HTTP; Service wuxuu haystaa sharciga ganacsiga; Repository wuxuu la hadlaa database-ka.

Configuration

src/main/resources/application.yml

```
spring:
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/nutritrack}
    username: ${DB_USER:nutri}
    password: ${DB_PASSWORD:nutri}
  jpa:
    hibernate:
      ddl-auto: validate      # Flyway ayaa abuura tables, Hibernate kaliya ayuu
    open-in-view: false
  flyway:
    enabled: true

server:
  forward-headers-strategy: framework

management:
  endpoints:
    web:
      exposure:
        include: health

app:
  jwt:
    secret: ${JWT_SECRET}     # base64, ugu yaraan 32 byte
    expiration-ms: 86400000   # 24 saac
```

```
# Ku bilow database-ka maxalliga ah
docker run --name nutri-db -e POSTGRES_DB=nutritrack -e POSTGRES_USER=nutri \
  -e POSTGRES_PASSWORD=nutri -p 5432:5432 -d postgres:16-alpine

# Samee furaha JWT
openssl rand -base64 32

# Ku orod
export JWT_SECRET="XXXX_halkan_ku_dhig"
./mvnw spring-boot:run
```

./mvnw spring-boot:run

> **HUBI**
>
> - Aad `http://localhost:8080/actuator/health` . Waa inaad aragto `{"status":"UP"}` .
> - Ku push GitHub: commit koowaad `chore: init project` .

☐ Waan dhameeyay

QAYBTA B · SPRING BOOT · 1.5 USBUUC

9. Database, Flyway iyo JPA/Hibernate

Hadaf: tables-ka oo ku abuuran migration, entities-ka oo ku xiran, repositories oo ku shaqeeya.

Flyway migration

Migration kasta waa fayl SQL ah oo la tirin karin marka la socodsiiyo. Isbeddel cusub = fayl cusub (`V2__...sql`).

src/main/resources/db/migration/V1__init.sql

```
CREATE TABLE users (
  id         BIGSERIAL PRIMARY KEY,
  email      VARCHAR(255) NOT NULL UNIQUE,
  password   VARCHAR(255) NOT NULL,
  full_name  VARCHAR(120) NOT NULL,
  role       VARCHAR(20)  NOT NULL
);

CREATE TABLE foods (
  id                BIGSERIAL PRIMARY KEY,
  name              VARCHAR(150) NOT NULL,
  calories_per_100g NUMERIC(7,2) NOT NULL CHECK (calories_per_100g >= 0),
  protein_g         NUMERIC(7,2) NOT NULL DEFAULT 0,
  carbs_g           NUMERIC(7,2) NOT NULL DEFAULT 0,
  fat_g             NUMERIC(7,2) NOT NULL DEFAULT 0
);
CREATE INDEX idx_foods_name ON foods (LOWER(name));
```

```
CREATE TABLE food_entries (
  id        BIGSERIAL PRIMARY KEY,
  user_id   BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  food_id   BIGINT NOT NULL REFERENCES foods(id),
  grams     NUMERIC(8,2) NOT NULL CHECK (grams > 0),
  meal_type VARCHAR(20) NOT NULL,
  eaten_on  DATE NOT NULL
);
CREATE INDEX idx_entries_user_date ON food_entries (user_id, eaten_on);

CREATE TABLE water_logs (
  id        BIGSERIAL PRIMARY KEY,
  user_id   BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  ml        INT NOT NULL CHECK (ml > 0),
  logged_on DATE NOT NULL
);
CREATE INDEX idx_water_user_date ON water_logs (user_id, logged_on);
```

V2__seed_foods.sql

```
INSERT INTO foods (name, calories_per_100g, protein_g, carbs_g, fat_g) VALUES
 ('Bariis cad (la kariyey)', 130, 2.7, 28, 0.3),
 ('Bur (rooti)',            265, 9.0, 49, 3.2),
 ('Digaag (naaska)',        165, 31,  0,  3.6),
 ('Bur-bur (moos)',          89, 1.1, 23, 0.3),
 ('Ukun (la kariyey)',      155, 13,  1.1, 11);
```

Entities

entity/User.java

```
@Entity
@Table(name = "users")
@Getter @Setter @NoArgsConstructor
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;

    @Column(name = "full_name", nullable = false)
```

```
    @Column(name = "full_name", nullable = false)
    private String fullName;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role = Role.USER;
}

public enum Role { USER, ADMIN }
public enum MealType { BREAKFAST, LUNCH, DINNER, SNACK }
```

entity/Food.java

```
@Entity
@Table(name = "foods")
@Getter @Setter @NoArgsConstructor
public class Food {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(name = "calories_per_100g", nullable = false)
    private BigDecimal caloriesPer100g;

    @Column(name = "protein_g", nullable = false)
    private BigDecimal proteinG = BigDecimal.ZERO;

    @Column(name = "carbs_g", nullable = false)
    private BigDecimal carbsG = BigDecimal.ZERO;

    @Column(name = "fat_g", nullable = false)
    private BigDecimal fatG = BigDecimal.ZERO;
}
```

entity/FoodEntry.java

```
@Entity
@Table(name = "food_entries")
@Getter @Setter @NoArgsConstructor
public class FoodEntry {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
```

```
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "food_id")
    private Food food;

    @Column(nullable = false)
    private BigDecimal grams;

    @Enumerated(EnumType.STRING)
    @Column(name = "meal_type", nullable = false)
    private MealType mealType;

    @Column(name = "eaten_on", nullable = false)
    private LocalDate eatenOn;
}
```

`WaterLog` waa sidaas oo kale (`user` , `ml` , `loggedOn`). Samee naftaada.

Repositories

`repository/*.java`

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}

public interface FoodRepository extends JpaRepository<Food, Long> {
    Page<Food> findByNameContainingIgnoreCase(String q, Pageable pageable);
}

public interface FoodEntryRepository extends JpaRepository<FoodEntry, Long> {
    @Query("select fe from FoodEntry fe join fetch fe.food "
         + "where fe.user.id = :userId and fe.eatenOn = :date")
    List<FoodEntry> findDaily(@Param("userId") Long userId, @Param("date") Local

    Optional<FoodEntry> findByIdAndUserId(Long id, Long userId);
}

public interface WaterLogRepository extends JpaRepository<WaterLog, Long> {
    @Query("select coalesce(sum(w.ml), 0) from WaterLog w "
         + "where w.user.id = :userId and w.loggedOn = :date")
    int totalMl(@Param("userId") Long userId, @Param("date") LocalDate date);
}
```

FAHMO

- **LAZY vs EAGER:** LAZY wuxuu xogta xiriirka ah soo qaadaa marka loo baahdo kaliya. Ka digtoonow **N+1 problem**; `join fetch` ayaa xalka ah.
- Ha soo celin entity si toos ah API-ga; isticmaal DTO (module xiga).
- Relationships oo kale: `@OneToOne` , `@OneToMany(mappedBy=...)` , `@ManyToMany` . Tusaale: User ↔ FoodEntry waa One-To-Many.

> LAYLI
>
> - Orod app-ka: Flyway waa inuu abuuraa tables-ka, Hibernate (`validate`) waa inuu aqbalaa. Haddii error yimaado, akhri farriinta, waxay kuu sheegaysaa column-ka khaldan.
> - Ku dar `V3__add_goal.sql` : column `daily_calorie_goal INT` oo ku jira `users` , kadibna ku dar entity-ga.

☐ Waan dhameeyay

QAYBTA B · SPRING BOOT · 2 USBUUC

10. REST API: DTO, Service, Controller

Hadaf: CRUD buuxa oo cuntooyinka ah, diiwaanka cuntada, biyaha iyo soo koobid maalinle ah.

REST xeerar

METHOD	ROUTE	WAXA UU SAMEEYO	STATUS
GET	/api/foods?q=bariis&page=0&size=20	Raadin + pagination	200
GET	/api/foods/{id}	Hal cunto	200 / 404
POST	/api/foods	Cunto cusub (ADMIN)	201
PUT	/api/foods/{id}	Cusboonaysii (ADMIN)	200
DELETE	/api/foods/{id}	Tir (ADMIN)	204

DELETE	/api/foods/{id}	Tir (ADMIN)	204
POST	/api/entries	Diiwaangeli cunto	201
DELETE	/api/entries/{id}	Tir diiwaankaaga	204
POST	/api/water	Diiwaangeli biyo	201
GET	/api/summary?date=2026-10-12	Soo koobid maalinle ah	200

DTO-yada (records)

dto/FoodRequest.java iyo FoodResponse.java

```
public record FoodRequest(
    @NotBlank @Size(max = 150) String name,
    @NotNull @DecimalMin("0") BigDecimal caloriesPer100g,
    @NotNull @DecimalMin("0") BigDecimal proteinG,
    @NotNull @DecimalMin("0") BigDecimal carbsG,
    @NotNull @DecimalMin("0") BigDecimal fatG) {}

public record FoodResponse(Long id, String name, BigDecimal caloriesPer100g,
                           BigDecimal proteinG, BigDecimal carbsG, BigDecimal fa
    public static FoodResponse from(Food f) {
        return new FoodResponse(f.getId(), f.getName(), f.getCaloriesPer100g(),
                                f.getProteinG(), f.getCarbsG(), f.getFatG());
    }
}
```

Service

service/FoodService.java

```
@Service
@RequiredArgsConstructor
public class FoodService {
    private final FoodRepository repo;

    @Transactional(readOnly = true)
    public Page<FoodResponse> search(String q, Pageable pageable) {
        return repo.findByNameContainingIgnoreCase(q, pageable).map(FoodResponse
    }

    @Transactional(readOnly = true)
    public FoodResponse get(Long id) {
```

```
        return FoodResponse.from(find(id));
    }

    @Transactional
    public FoodResponse create(FoodRequest r) {
        Food f = new Food();
        apply(f, r);
        return FoodResponse.from(repo.save(f));
    }

    @Transactional
    public FoodResponse update(Long id, FoodRequest r) {
        Food f = find(id);
        apply(f, r);
        return FoodResponse.from(f);       // dirty checking ayaa kaydinaya
    }

    @Transactional
    public void delete(Long id) {
        repo.delete(find(id));
    }

    private Food find(Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Cunto " + id + " l
    }

    private void apply(Food f, FoodRequest r) {
        f.setName(r.name());
        f.setCaloriesPer100g(r.caloriesPer100g());
        f.setProteinG(r.proteinG());
        f.setCarbsG(r.carbsG());
        f.setFatG(r.fatG());
    }
}
```

Controller

controller/FoodController.java

```
@RestController
@RequestMapping("/api/foods")
@RequiredArgsConstructor
public class FoodController {
    private final FoodService service;
```

```
    private final FoodService service;

    @GetMapping
    public Page<FoodResponse> list(
            @RequestParam(defaultValue = "") String q,
            @PageableDefault(size = 20, sort = "name") Pageable pageable) {
        return service.search(q, pageable);
    }

    @GetMapping("/{id}")
    public FoodResponse get(@PathVariable Long id) {
        return service.get(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    @PreAuthorize("hasRole('ADMIN')")
    public FoodResponse create(@Valid @RequestBody FoodRequest body) {
        return service.create(body);
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public FoodResponse update(@PathVariable Long id, @Valid @RequestBody FoodRe
        return service.update(id, body);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasRole('ADMIN')")
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}
```

Diiwaanka cuntada iyo soo koobidda

service/SummaryService.java

```
public record DailySummary(LocalDate date, BigDecimal calories, BigDecimal prote
                           BigDecimal carbsG, BigDecimal fatG, int waterMl) {}

@Service
@RequiredArgsConstructor
public class SummaryService {
    private final FoodEntryRepository entries;
```

```
    private final FoodEntryRepository entries;
    private final WaterLogRepository water;
    private final UserRepository users;

    @Transactional(readOnly = true)
    public DailySummary forDay(String email, LocalDate date) {
        Long userId = users.findByEmail(email).orElseThrow().getId();
        List<FoodEntry> list = entries.findDaily(userId, date);

        BigDecimal cal = sum(list, e -> e.getFood().getCaloriesPer100g(), e);
        BigDecimal pro = sum(list, e -> e.getFood().getProteinG(), e);
        BigDecimal carb = sum(list, e -> e.getFood().getCarbsG(), e);
        BigDecimal fat = sum(list, e -> e.getFood().getFatG(), e);
        return new DailySummary(date, cal, pro, carb, fat, water.totalMl(userId,
    }

    // qiimaha per-100g * grams / 100
    private BigDecimal sum(List<FoodEntry> list, Function<FoodEntry, BigDecimal>
        return list.stream()
            .map(e -> per100.apply(e).multiply(e.getGrams())
                            .divide(BigDecimal.valueOf(100), 2, RoundingMode.HAL
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}
```

Cabbirka `ignored` waa qasab ma aha; waxaad ka saari kartaa oo hagaajin kartaa naftaada. Waa layli dib-u-habayn (refactoring) oo wanaagsan.

`controller/SummaryController.java`

```
@RestController
@RequestMapping("/api/summary")
@RequiredArgsConstructor
public class SummaryController {
    private final SummaryService service;

    @GetMapping
    public DailySummary day(@AuthenticationPrincipal UserDetails principal,
                            @RequestParam(required = false)
                            @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) Local
        return service.forDay(principal.getUsername(), date != null ? date : Loc
    }
}
```

LAYLI: DHAMMAYSTIR ADIGU

- `EntryController` + `EntryService`: `POST /api/entries` (body: `foodId, grams, mealType, eatenOn`) iyo `DELETE`. Isticmaal `findByIdAndUserId` si isticmaale uusan tirin diiwaanka qof kale.
- `WaterController`: `POST /api/water` (`ml`).
- Ku dar `GET /api/entries?date=...` oo soo celiya liiska maalinta.
- Tijaabi dhammaan Postman ama Swagger UI: `http://localhost:8080/swagger-ui.html`.

☐ Waan dhameeyay

QAYBTA B · SPRING BOOT · 1 USBUUC

11. Validation, Error Handling iyo Logging

Hadaf: API-gaagu waa inuu jawaabaa si joogto ah marka xogtu khaldan tahay ama wax la waayo.

`exception/GlobalExceptionHandler.java`

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) { super(message); }
}

@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    ProblemDetail notFound(ResourceNotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMess
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail invalid(MethodArgumentNotValidException ex) {
        var pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Xogta
        pd.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
            .collect(Collectors.toMap(FieldError::getField,
                                      fe -> String.valueOf(fe.getDefaultMessage(
```

```
                                        (a, b) -> a)));
        return pd;
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    ProblemDetail conflict(DataIntegrityViolationException ex) {
        log.warn("Data conflict: {}", ex.getMostSpecificCause().getMessage());
        return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, "Xogtan hor
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail unexpected(Exception ex) {
        log.error("Khalad lama filaan ah", ex);
        return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR
                                                "Cilad ayaa dhacday. Fadlan isku
    }
}
```

Hadda `POST /api/foods` oo magac maran leh wuxuu soo celinayaa `400` iyo liiska fields-ka khaldan, halkii uu ku soo laaban lahaa stack trace.

Logging

```
@Slf4j
@Service
public class EntryService {
    public void add(...) {
        log.info("Diiwaan cusub: user={}, foodId={}", userId, foodId);
        log.debug("Faahfaahin dheeraad ah");
    }
}
```

```
logging:
  level:
    root: INFO
    com.nutritrack: DEBUG    # development; production ku dhig INFO
```

- Heerarka: `ERROR`, `WARN`, `INFO`, `DEBUG`, `TRACE`.
- **Weligaa ha log gareyn** password-ka, token-ka ama xogta shakhsi.

> **LAYLI**
>
> - Ku dar `@Valid` dhammaan request-yada, oo hubi in `grams <= 0` uu soo celiyo

- Ku dar `@Valid` dhammaan request yada, oo hubi in `grams > 0` ama sidoo kale 400.
- Ku dar `@NotBlank` `@Email` email-ka diiwaangelinta.

☐ Waan dhameeyay

QAYBTA B · SPRING BOOT · 2 USBUUC

12. Authentication iyo Authorization (JWT)

Hadaf: isticmaaluhu wuxuu is diiwaangeliyaa, login ku sameeyaa, token ayuu helaa, oo token-kaas ayaa ilaaliya routes-ka.

Hab-socodka

1. `POST /api/auth/register` → password waa la hash-garaynayaa (BCrypt) oo la kaydiyaa.
2. `POST /api/auth/login` → email + password waa la hubiyaa, token JWT ayaa la soo celiyaa.
3. Codsi kasta oo kale wuxuu wataa `Authorization: Bearer <token>`.
4. Filter-ku wuxuu hubiyaa token-ka oo dejiyaa isticmaalaha SecurityContext-ka.

Authentication = yaad tahay. **Authorization** = maxaad sameyn kartaa (doorarka USER/ADMIN).

security/CustomUserDetailsService.java

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository users;

    @Override
    public UserDetails loadUserByUsername(String email) {
        com.nutritrack.entity.User u = users.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException("Isticmaale lama he
        return org.springframework.security.core.userdetails.User
            .withUsername(u.getEmail())
            .password(u.getPassword())
            .authorities("ROLE_" + u.getRole().name())
```

```
            .authorities("ROLE_" + u.getRole().name())
            .build();
    }
}
```

security/JwtService.java

```
@Service
public class JwtService {
    @Value("${app.jwt.secret}") private String secret;
    @Value("${app.jwt.expiration-ms}") private long expirationMs;

    private SecretKey key() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
    }

    public String generateToken(UserDetails user) {
        Date now = new Date();
        return Jwts.builder()
            .subject(user.getUsername())
            .issuedAt(now)
            .expiration(new Date(now.getTime() + expirationMs))
            .signWith(key())
            .compact();
    }

    /** Waxay soo celisaa email-ka, ama null haddii token-ku khaldan yahay ama d
    public String validateAndGetUsername(String token) {
        try {
            return Jwts.parser().verifyWith(key()).build()
                .parseSignedClaims(token).getPayload().getSubject();
        } catch (JwtException | IllegalArgumentException e) {
            return null;
        }
    }
}
```

security/JwtAuthFilter.java

```
@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService jwt;
    private final CustomUserDetailsService users;
```

```
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse
                                    FilterChain chain) throws ServletException,
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String email = jwt.validateAndGetUsername(header.substring(7));
            if (email != null && SecurityContextHolder.getContext().getAuthentic
                UserDetails details = users.loadUserByUsername(email);
                var auth = new UsernamePasswordAuthenticationToken(
                        details, null, details.getAuthorities());
                auth.setDetails(new WebAuthenticationDetailsSource().buildDetail
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

config/SecurityConfig.java

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())     // API stateless ah, cookie ma ist
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolic
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/auth/**", "/actuator/health",
                                 "/swagger-ui/**", "/swagger-ui.html", "/v3/api-
                .anyRequest().authenticated())
            .exceptionHandling(e -> e
                .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UN
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
```

```
    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration cfg)
        return cfg.getAuthenticationManager();
    }
}
```

service/AuthService.java iyo controller/AuthController.java

```
public record RegisterRequest(@NotBlank @Email String email,
                              @NotBlank @Size(min = 8, max = 72) String password
                              @NotBlank @Size(max = 120) String fullName) {}
public record LoginRequest(@NotBlank @Email String email, @NotBlank String passw
public record AuthResponse(String token) {}

@Service
@RequiredArgsConstructor
public class AuthService {
    private final UserRepository users;
    private final PasswordEncoder encoder;
    private final AuthenticationManager authManager;
    private final CustomUserDetailsService detailsService;
    private final JwtService jwt;

    @Transactional
    public AuthResponse register(RegisterRequest r) {
        if (users.existsByEmail(r.email()))
            throw new DataIntegrityViolationException("Email horay ayaa loo isti
        User u = new User();
        u.setEmail(r.email());
        u.setFullName(r.fullName());
        u.setPassword(encoder.encode(r.password()));
        users.save(u);
        return new AuthResponse(jwt.generateToken(detailsService.loadUserByUsern
    }

    public AuthResponse login(LoginRequest r) {
        authManager.authenticate(new UsernamePasswordAuthenticationToken(r.email
        return new AuthResponse(jwt.generateToken(detailsService.loadUserByUsern
    }
}

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
```

```
public class AuthController {
    private final AuthService service;

    @PostMapping("/register")
    @ResponseStatus(HttpStatus.CREATED)
    public AuthResponse register(@Valid @RequestBody RegisterRequest r) { return

    @PostMapping("/login")
    public AuthResponse login(@Valid @RequestBody LoginRequest r) { return servi
}
```

Ku dar handler ee `AuthenticationException` `GlobalExceptionHandler` -ka oo soo celi `401` markii password-ku khaldan yahay (haddii kale wuxuu u dhici doonaa 500 ee `Exception.class`).

```
@ExceptionHandler(AuthenticationException.class)
ProblemDetail badCredentials(AuthenticationException ex) {
    return ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, "Email ama
}
```

Tijaabi

```
curl -X POST localhost:8080/api/auth/register -H "Content-Type: application/json
  -d '{"email":"mo@example.com","password":"Password123","fullName":"Mohamed"}'

curl localhost:8080/api/foods -H "Authorization: Bearer TOKEN_HALKAN"
```

Samee ADMIN-ka koowaad: ku beddel role-ka database-ka gacanta (`UPDATE users SET role='ADMIN' WHERE email='...'`), ama ku samee migration gaar ah oo aan lagu soo daabicin production.

> **LAYLI**
>
> - Hubi: `POST /api/foods` oo token USER leh = 403, token ADMIN leh = 201, token la'aan = 401.
> - Ku dar endpoint `GET /api/me` oo soo celiya email-ka iyo role-ka.

Waan dhameeyay

QAYBTA B · SPRING BOOT · 1.5 USBUUC

13. Testing

Hadaf: inaad ku kalsoonaato isbeddel kasta adigoo isticmaalaya tests otomaatig ah.

NOOCA	WAXA LA TIJAABIYAA	QALAB
Unit	Class keliya, wax kale la mock-gareeyo	JUnit 5, Mockito
Integration	Controller → DB dhab ah	@SpringBootTest, MockMvc, Testcontainers

src/test/java/.../service/FoodServiceTest.java

```
@ExtendWith(MockitoExtension.class)
class FoodServiceTest {
    @Mock FoodRepository repo;
    @InjectMocks FoodService service;

    @Test
    void get_throwsWhenMissing() {
        when(repo.findById(99L)).thenReturn(Optional.empty());

        assertThrows(ResourceNotFoundException.class, () -> service.get(99L));
    }

    @Test
    void create_savesAndReturnsResponse() {
        var req = new FoodRequest("Bariis", new BigDecimal("130"),
                new BigDecimal("2.7"), new BigDecimal("28"), new BigDecimal("0.3
        when(repo.save(any(Food.class))).thenAnswer(inv -> {
            Food f = inv.getArgument(0);
            f.setId(1L);
            return f;
        });

        FoodResponse res = service.create(req);

        assertEquals(1L, res.id());
        assertEquals("Bariis", res.name());
    }
```

```
    }
}
```

AuthFlowIT.java

```
@SpringBootTest
@AutoConfigureMockMvc
@Testcontainers
class AuthFlowIT {
    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16-al

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("app.jwt.secret", () -> "dGVzdC1zZWNyZXQta2V5LWZvci10ZXN0cy0xMjM0N
    }

    @Autowired MockMvc mvc;

    @Test
    void protectedRouteNeedsToken_thenWorksWithToken() throws Exception {
        mvc.perform(get("/api/foods")).andExpect(status().isUnauthorized());

        String body = """
            {"email":"test@example.com","password":"Password123","fullName":"Tes
        String json = mvc.perform(post("/api/auth/register")
                .contentType(MediaType.APPLICATION_JSON).content(body))
            .andExpect(status().isCreated())
            .andReturn().getResponse().getContentAsString();

        String token = new ObjectMapper().readTree(json).get("token").asText();

        mvc.perform(get("/api/foods").header("Authorization", "Bearer " + token)
            .andExpect(status().isOk());
    }
}
```

Testcontainers wuxuu u baahan yahay Docker oo socda. Orod: `./mvnw verify`. Hadaf: dhammaan **service logic** iyo dhammaan routes-ka muhiimka ah oo leh test.

> **LAYLI**
>
> - Qor test `SummaryService` oo hubinaya xisaabta: 200g cunto oo 130 kal/100g =

260.

- Qor integration test: USER wuu helaa 403 marka uu isku dayo `POST /api/foods`.

☐ Waan dhameeyay

QAYBTA C · PRODUCTION · 1.5 USBUUC

14. Docker iyo Linux

Hadaf: app-kaagu wuxuu ku shaqeeyaa meel kasta si isku mid ah, kaas oo ah container.

`Dockerfile`

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY .mvn .mvn
COPY mvnw pom.xml ./
RUN ./mvnw -q -B dependency:go-offline
COPY src src
RUN ./mvnw -q -B -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
RUN useradd -r -u 1001 appuser
COPY --from=build /app/target/*.jar app.jar
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

`.dockerignore`

```
target
.git
.idea
.env
```

`compose.yaml (development: build maxalli ah)`

```
services:
  db:
```

```
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: nutritrack
      POSTGRES_USER: nutri
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U nutri -d nutritrack"]
      interval: 5s
      timeout: 3s
      retries: 10

  app:
    build: .
    depends_on:
      db:
        condition: service_healthy
    environment:
      DB_URL: jdbc:postgresql://db:5432/nutritrack
      DB_USER: nutri
      DB_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
    ports:
      - "8080:8080"
    restart: unless-stopped

volumes:
  pgdata:
```

```
.env (ha ku rin Git)

DB_PASSWORD=bedel-kan-password-adag
JWT_SECRET=ku-dhig-natiijada-openssl-rand-base64-32
```

```
docker compose up --build -d
docker compose ps
docker compose logs -f app
docker compose down
```

Linux aasaas ah (server-ka)

```
ssh user@SERVER_IP
sudo apt update && sudo apt upgrade -y
ls -la
```

```
ls -la
cat file.txt
nano file.txt
tail -f /var/log/nginx/error.log
systemctl status nginx
sudo ufw allow OpenSSH && sudo ufw allow 80 && sudo ufw allow 443 && sudo ufw en
chmod 600 .env
df -h        # meel kaydka
free -h      # RAM
top          # hawlaha socda
```

HUBI

- `docker compose up` kadib, `curl localhost:8080/actuator/health` waa UP.
- Orod `docker compose down` oo mar kale `up` : xogtu weli way ku jirtaa (volume).

☐ Waan dhameeyay

QAYBTA C · PRODUCTION · 1 USBUUC

15. CI/CD (GitHub Actions)

Hadaf: marka aad push gareyso, tests ayaa socda, image-ka ayaa la dhisaa, server-ka ayaa is cusboonaysiiya.

`.github/workflows/ci-cd.yml`

```
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        ...
```

```
        with:
          distribution: temurin
          java-version: 21
          cache: maven
      - run: ./mvnw -B verify

  publish:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${{ github.repository_owner }}/nutritrack:latest

  deploy:
    needs: publish
    runs-on: ubuntu-latest
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          script: |
            cd ~/nutritrack
            docker compose pull app
            docker compose up -d
            docker image prune -f
```

- Secrets: GitHub repo → Settings → Secrets and variables → Actions. Ku dar `SERVER_HOST`, `SERVER_USER`, `SERVER_SSH_KEY` (furaha gaarka ah ee SSH).
- Ha gelin password-ka ama JWT_SECRET workflow-ga; waxay ku jiraan `.env` ee server-ka.
- Package-ka GHCR ka dhig **public** ama server-ka ku samee `docker login ghcr.io`.

- Package-ka GHCR ka dhig **public**, ama server-ka ku samee `docker login ghcr.io` adigoo isticmaalaya Personal Access Token oo leh `read:packages` .

> **LAYLI**
>
> - Fur PR oo tests-ka ku dhaca: arag in GitHub uu mamnuucayo merge. Hagaaji, kadibna merge.
> - Ku dar badge CI `README.md` -gaaga.

☐ Waan dhameeyay

QAYBTA C · PRODUCTION · 1.5 USBUUC

16. Deployment: server, domain iyo SSL

Hadaf: NutriTrack oo ku shaqeeya `https://api.domain-kaaga.com` .

Dariiqa A: VPS (waxbarasho ugu fiican)

1. **Kireyso VPS** Ubuntu 24.04 (Hetzner, DigitalOcean, AWS EC2/Lightsail, ama Oracle Cloud free tier). 1–2 GB RAM ayaa ku filan.
2. **Ammaan:** samee user aan root ahayn, SSH key kaliya, firewall (`ufw`).
3. **Ku rakib Docker** (docs.docker.com/engine/install/ubuntu).
4. **Domain:** iibso domain (Namecheap, Cloudflare, iwm). Samee **A record**: `api` → IP-ga server-ka.
5. **Geli files-ka:** `~/nutritrack/compose.yaml` iyo `.env` .

`~/nutritrack/compose.yaml (production)`

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: nutritrack
      POSTGRES_USER: nutri
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
```

```
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U nutri -d nutritrack"]
      interval: 5s
      timeout: 3s
      retries: 10

  app:
    image: ghcr.io/USERNAME/nutritrack:latest
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    environment:
      DB_URL: jdbc:postgresql://db:5432/nutritrack
      DB_USER: nutri
      DB_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
    ports:
      - "127.0.0.1:8080:8080"     # kaliya server-ka gudihiisa; Nginx ayaa dibad

volumes:
  pgdata:
```

Nginx (reverse proxy)

```
sudo apt install -y nginx certbot python3-certbot-nginx
```

```
/etc/nginx/sites-available/nutritrack

server {
    listen 80;
    server_name api.domain-kaaga.com;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/nutritrack /etc/nginx/sites-enabled/
```

```
sudo nginx -t && sudo systemctl reload nginx

# SSL bilaash ah (Let's Encrypt)
sudo certbot --nginx -d api.domain-kaaga.com
sudo certbot renew --dry-run     # hubi cusboonaysiinta otomaatiga ah
```

Orod oo hubi

```
cd ~/nutritrack
docker compose up -d
docker compose logs -f app

curl https://api.domain-kaaga.com/actuator/health
```

Dariiqa B: platform la maamulo (degdeg)

Render, Railway ama Fly.io: ku xir GitHub repo-gaaga, dooro Dockerfile, ku dar PostgreSQL maamulan, geli variables (`DB_URL` , `JWT_SECRET` ...). SSL iyo domain ayaa ku jira. Waa degdeg, laakiin waxba ma bartid server-ka; sidaas darteed dariiqa A marka hore ku samee ugu yaraan hal mar.

Liiska hubinta production

- Password-ka database iyo JWT_SECRET waa kuwo adag oo ku jira `.env` keliya.
- Port-ka database (5432) dibadda looma furin.
- Backup: `docker exec ... pg_dump` + cron, ama backup-ka provider-ka.
- CORS: haddii frontend uu jiro, u oggolow domain-kiisa kaliya.
- Monitoring: `/actuator/health` oo uu hubiyo UptimeRobot.

> **HUBI**
>
> - Push gareey isbeddel yar → GitHub Actions → server-ka wuu is cusboonaysiiyaa.
> - Swagger UI wuxuu ka shaqeynayaa `https://api.domain-kaaga.com/swagger-ui.html` .

■ Waan dhameeyay

QAYBTA D · XIRFAD · 1 USBUUC

17. Design Patterns iyo Architecture

Hadaf: inaad aqoonsato patterns-ka caadiga ah, oo aad ka jawaabi karto su'aalaha interview-ka.

PATTERN	WAA MAXAY	TUSAALE
Singleton	Hal instance oo keliya	Spring beans (default scope)
Factory	Hal meel ayaa abuura objects	`List.of()` , `BeanFactory`
Builder	Abuur object adag tallaabo-tallaabo	`Jwts.builder()` , Lombok `@Builder`
Strategy	Algorithm la beddeli karo	`PasswordEncoder` (BCrypt, Argon2...)
Observer	Dhacdo → dhegeystayaal badan	Spring `ApplicationEvent`
Proxy	Wakiil ka hortaga object	`@Transactional` wuxuu isticmaalaa proxy

```
// Strategy: qaabka lagu xisaabiyo bartilmaameedka kalooriyada
interface CalorieGoalStrategy { int dailyGoal(int weightKg, int heightCm, int ag

class MifflinStJeor implements CalorieGoalStrategy {
    public int dailyGoal(int w, int h, int age) {
        return (int) (10 * w + 6.25 * h - 5 * age + 5);   // ragga, aasaas
    }
}
```

SOLID (kooban)

- **S**: class kasta hal mas'uuliyad.
- **O**: fur kordhinta, xidh beddelka.
- **L**: subclass waa inuu bedeli karaa parent-kiisa.
- **I**: interface yaryar oo gaar ah.
- **D**: ku tiirsanow abstraction, ma aha class dhab ah (Spring DI).

Architecture

- **Layered/MVC:** waxa aad dhistay. Waa ku filan 90% mashaariicda.
- **Clean/Hexagonal:** ganacsiga oo ka madax-bannaan framework-ka.

- **Microservices:** baro kadib marka aad sifiican u fahanto monolith; ha ku bilaabin.

☐ Waan dhameeyay

QAYBTA D · XIRFAD · 1 USBUUC IYO KA DAMBEYSA

18. Portfolio, CV iyo Shaqo raadin

Hadaf: inaad muujiso shaqadaada, oo aad hesho shaqo ama freelance gig.

README.md oo wanaagsan

- Sharaxaad kooban (2 jumlo) iyo link live ah.
- Tech stack: Java 21, Spring Boot, PostgreSQL, Docker, GitHub Actions.
- Sawir architecture-ka ah iyo ERD.
- Sida loo orodo maxalli ahaan (`docker compose up`).
- Liiska endpoints (ama Swagger link) iyo Postman collection.
- Badge-ka CI.

Mashaariic kale oo aad dhisi karto

- **School Management:** students, teachers, courses, grades, attendance (relationships adag).
- **E-Commerce:** products, cart, orders, payment integration (transactions, stock).
- **Hospital / Appointment System:** booking, doctors schedule, conflicts.
- Ku dar: file upload, email, scheduled jobs (`@Scheduled`), caching (`@Cacheable`), frontend yar (React ama HTML).

Su'aalaha interview-ka caadiga ah

- Farqiga `HashMap` iyo `ConcurrentHashMap` ?
- Sidee u shaqeeyaa `equals/hashCode` ?
- Waa maxay Dependency Injection, iyo faa'iidadeeda?
- Farqiga `@Component` , `@Service` , `@Repository` ?
- Waa maxay N+1 problem, sidee loo xalliyaa?
- Sidee u shaqeeyaa JWT? Maxay tahay khatarta in la dhigo token `localStorage` ?

- `@Transactional` sidee u shaqeeyaa, goorma ayuusan shaqayn?
- Sharax Docker image iyo container.

Tallaabooyinka ugu dambeeya

1. Nadiifi GitHub-kaaga: profile README, 3 repo oo wanaagsan, commits joogto ah.
2. CV hal bog: xirfado, 3 mashruuc oo leh link live, waxbarasho.
3. LinkedIn: ku qor "Java Backend Developer", la xiriir shirkado iyo recruiters.
4. Codso 5 shaqo toddobaad kasta, oo sii wad dhisidda mashruuc cusub.
5. Freelance: Upwork, Fiverr, shirkadaha maxalliga ah; ku bilow shaqooyin yar (API yar, bug fixes).

Sii wad barashada: Spring Cloud/Microservices, Kafka/RabbitMQ, Redis, Kubernetes, AWS, iyo algorithms (LeetCode).

☐ Waan dhameeyay

DHAMAAD

Marka aad dhammayso

Waxaad yeelan doontaa Java oo adag, Spring Boot, database, security, testing, Docker, CI/CD, iyo mashruuc ku shaqeeya internetka. Taasi waa heerka Junior Java Developer ee diyaarka u ah shaqo. Waqti ku fiican, mashruuc ka dib mashruuc.

Haddii aad rabto, waxaan kuu samayn karaa **module kasta** oo aad rabto in si xoog leh loo sharaxo (tusaale JPA relationships, JWT, Docker), ama waxaan kuu diyaarin karaa **School Management** ama **E-Commerce** sidoo kale.